



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

**Факультет «Информатика и системы управления»
Кафедра «Системы обработки информации и управления»**

**Отчет по рубежному контролю №1
по дисциплине «Методы машинного обучения в АСОИУ»
Вариант №3, 23**

**Выполнил:
студент группы ИУ5Ц-21М
Перевощиков Н.Д.
подпись, дата**

**Проверил:
к.т.н., доц., Гапанюк Ю.Е.
подпись, дата**

2026 г.

СОДЕРЖАНИЕ

1. Требования по группам.....	3
2. Дополнительные требования по группам	3
3. Условия задач.....	3
4. Ход работы (листинг).....	4

1. Требования по группам

Для студентов групп ИУ5-21М, ИУ5-22М, ИУ5-23М, ИУ5-24М номер варианта = номер в списке группы.

2. Дополнительные требования по группам

Для студентов групп ИУ5-21М, ИУ5И-21М - для пары произвольных колонок данных построить график "Диаграмма рассеяния".

Каждая задача предполагает использование набора данных. Набор данных выбирается Вами произвольно с учетом следующих условий:

- Вы можете использовать один набор данных для решения всех задач, или решать каждую задачу на своем наборе данных.
- Набор данных должен отличаться от набора данных, который использовался в лекции для решения рассматриваемой задачи.
- Вы можете выбрать произвольный набор данных (например тот, который Вы использовали в лабораторных работах) или создать собственный набор данных (что актуально для некоторых задач, например, для задач удаления псевдоконстантных или повторяющихся признаков).
- Выбранный или созданный Вами набор данных должен удовлетворять условиям поставленной задачи. Например, если решается задача устранения пропусков, то набор данных должен содержать пропуски.

Наборы данных

Набор данных о выживаемости уличных торговцев едой в городах (Индия): <https://www.kaggle.com/datasets/sinanshereef/urban-street-food-vendor-survival-dataset-india>

3. Условия задач

Номер задачи №1 (Задача №3)

Для набора данных проведите кодирование одного (произвольного) категориального признака с использованием метода "weight of evidence (WoE) encoding".

Номер задачи №2 (Задача №23)

Для набора данных для одного (произвольного) числового признака проведите обнаружение и удаление выбросов на основе правила трех сигм.

4. Ход работы (листинг)

Для начала импортируем необходимые библиотеки и получаем информацию о датасете.

```
# Импортируем необходимые библиотеки
import pandas as pd
import numpy as np

# Загрузка данных
df = pd.read_csv('urban_street_food_vendor_survival_dataset.csv')

# Общая информация о датасете
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22000 entries, 0 to 21999
Data columns (total 20 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   vendor_id                            22000 non-null  object
 1   city                                  22000 non-null  object
 2   zone_type                             22000 non-null  object
 3   vendor_age_years                     21242 non-null  float64
 4   years_in_business                   20963 non-null  float64
 5   food_category                        22000 non-null  object
 6   license_status                       21350 non-null  object
 7   avg_daily_revenue_inr                20062 non-null  float64
 8   avg_daily_customers                  20747 non-null  float64
 9   monthly_stall_rent_inr               20890 non-null  float64
10  num_helpers                           22000 non-null  int64
11  hours_open_per_day                   21558 non-null  float64
12  competition_within_100m              21116 non-null  float64
13  monthly_health_inspection_score      14241 non-null  float64
14  had_fine_last_year                   21779 non-null  float64
15  avg_monthly_rainfall_mm              22000 non-null  float64
16  season_of_observation                22000 non-null  object
17  has_online_presence                  21807 non-null  float64
18  customer_complaint_rate              19580 non-null  float64
19  vendor_survived                      22000 non-null  int64
dtypes: float64(12), int64(2), object(6)
memory usage: 3.4+ MB
```

```
# Первые 5 строк датасета
df.head()
```

	vendor_id	city	zone_type	vendor_age_years	years_in_business	food_category	license_status	avg_daily_revenue_inr	avg_daily_customers	monthly_stall_rent_inr	num_helpers	hours_open_per_day	competition_within_100m	monthly_health
0	VSF-00001	Delhi	Industrial	34.0	10.98	Chinese	Licensed	332.20	10.0	0.00	1	10.00	4.0	
1	VSF-00002	Kochi	University Area	33.0	5.53	Chaat	Licensed	3138.90	180.0	2112.10	1	6.42	3.0	
2	VSF-00003	Hyderabad	Industrial	48.0	2.70	Desserts & Sweets	Licensed	1626.36	67.0	5487.75	3	11.80	0.0	
3	VSF-00004	Bengaluru	Transit Hub	NaN	9.77	Beverages	Licensed	3592.98	171.0	1960.79	3	12.53	NaN	
4	VSF-00005	Delhi	Transit Hub	51.0	21.24	South Indian	Licensed	2418.07	105.0	2720.82	4	6.07	5.0	

Проверяем датасет на пропуски.

```
# Проверка на наличие пропусков
print("Проверка на наличие пропусков:")
df.isnull().sum()
```

```
Проверка на наличие пропусков:
vendor_id                0
city                    0
zone_type                0
vendor_age_years         758
years_in_business       1037
food_category            0
license_status           650
avg_daily_revenue_inr    1938
avg_daily_customers      1253
monthly_stall_rent_inr   1110
num_helpers              0
hours_open_per_day       442
competition_within_100m  884
monthly_health_inspection_score  7759
had_fine_last_year       221
avg_monthly_rainfall_mm   0
season_of_observation     0
has_online_presence       193
customer_complaint_rate  2420
vendor_survived           0
dtype: int64
```

Датасет имеет пропуски, поэтому проведем обработку пропусков. В этом коде выполняется очистка пропусков в датасете по двум типам признаков:

1. Категориальные столбцы (object) заполняются модой (наиболее частым значением), если она существует; иначе 'Unknown'.
2. Числовые столбцы (float64, int64). Для бинарных/счётных признаков (например, `had_fine_last_year`, `has_online_presence`) заполняются нулём.

Для остальных числовых заполняются медианой (устойчивой к выбросам).

В конце проверяется, что после обработки пропусков больше нет: `df.isnull().sum()` должен вернуть нули.

```

# 1. Для категориальных столбцов (object) - заполняем модой или 'Unknown'
cat_cols = df.select_dtypes(include=['object']).columns
for col in cat_cols:
    if df[col].isnull().sum() > 0:
        mode_val = df[col].mode()
        if len(mode_val) > 0:
            df[col] = df[col].fillna(mode_val.iloc[0])
        else:
            df[col] = df[col].fillna('Unknown')

# 2. Для числовых столбцов - заполняем медианой (или 0 для счётных/бинарных)
num_cols = df.select_dtypes(include=['float64', 'int64']).columns

# Отдельно выделим бинарные/счётные признаки, где 0 логичен:
binary_or_count_cols = [
    'num_helpers',
    'had_fine_last_year',
    'has_online_presence',
    'vendor_survived'
]

for col in num_cols:
    if df[col].isnull().sum() > 0:
        if col in binary_or_count_cols:
            df[col] = df[col].fillna(0)
        else:
            median_val = df[col].median()
            df[col] = df[col].fillna(median_val)

# Проверка: должны остаться 0 пропусков
print("Пропуски после очистки:")
df.isnull().sum()

```

Проверяем на наличие пропусков в датасете после обработки. После обработки пропусков все столбцы содержат нулевое количество пропусков.

```

Пропуски после очистки:
vendor_id          0
city               0
zone_type          0
vendor_age_years   0
years_in_business  0
food_category      0
license_status     0
avg_daily_revenue_inr  0
avg_daily_customers  0
monthly_stall_rent_inr  0
num_helpers        0
hours_open_per_day  0
competition_within_100m  0
monthly_health_inspection_score  0
had_fine_last_year  0
avg_monthly_rainfall_mm  0
season_of_observation  0
has_online_presence  0
customer_complaint_rate  0
vendor_survived    0
dtype: int64

```

Задача №3

Для набора данных проведем кодирование одного (произвольного) категориального признака с использованием метода "weight of evidence (WoE) encoding".

Была создана бинарная целевая переменная `high_revenue`, где 0 – средняя дневная выручка $\leq$ медианы (1516.59 INR) 54.40% объектов, 1 – средняя дневная выручка $>$ медианы 45.60% объектов.

```
# Бинарная целевая переменная
threshold = df['avg_daily_revenue_inr'].median()
df['high_revenue'] = (df['avg_daily_revenue_inr'] > threshold).astype(int)

print(f"Медиана средней дневной выручки: {threshold:.2f} INR")
print(df['high_revenue'].value_counts(normalize=True))
```

```
Медиана средней дневной выручки: 1516.59 INR
high_revenue
0    0.544045
1    0.455955
Name: proportion, dtype: float64
```

Выполняем WoE-кодирование категориального признака `city` (город) на основе бинарной целевой переменной `high_revenue` (высокая/низкая средняя дневная выручка).

1. Подсчитаны частоты городов — определены топ-10 городов по количеству записей.
2. Сгруппированы редкие города: города с ≤ 30 записями объединены в категорию "Other".
3. Вычислен WoE для каждой группы с добавлением smoothing ($\text{eps} = 0.5$) для стабильности. Для каждой группы:

$$\text{WoE} = \ln \left(\frac{\% \text{ не-событий}}{\% \text{ событий}} \right)$$

где событие = `high_revenue == 1`.

4. Создана карта соответствия `city` WoE, и признак `city_WoE` добавлен в датафрейм.

Получили новый числовой признак `city_WoE`, отражающий

ассоциацию каждого города с уровнем выручки. Например: Mumbai: +0.0080
слегка выше доля низкой выручки, Hyderabad: −0.0091 выше доля высокой
выручки.

```
# Подсчёт частот городов
city_counts = df['city'].value_counts()
print("Частоты городов (топ 10):")
print(city_counts.head(10))

# Определяем порог для "Other": например, меньше 30 записей, объединяем
min_count_for_keep = 30
common_cities = city_counts[city_counts >= min_count_for_keep].index.tolist()
df['city_group'] = df['city'].where(df['city'].isin(common_cities), 'Other')

# Вычисляем WoE с добавлением smoothing (eps = 0.5)
total_events = df['high_revenue'].sum()
total_non_events = len(df) - total_events
eps = 0.5

woe_data = (
    df.groupby('city_group')['high_revenue']
      .agg(count='count', events='sum')
      .reset_index()
)
woe_data['non_events'] = woe_data['count'] - woe_data['events']

woe_data['%_events'] = (woe_data['events'] + eps) / (total_events + eps)
woe_data['%_non_events'] = (woe_data['non_events'] + eps) / (total_non_events + eps)

woe_data['WoE'] = np.log(woe_data['%_non_events'] / woe_data['%_events'])

# Мappings
woe_map = dict(zip(woe_data['city_group'], woe_data['WoE']))
df['city_WoE'] = df['city_group'].map(woe_map)

Частоты городов (топ 10):
city
Mumbai      4392
Delhi        4042
Bengaluru    3279
Hyderabad    2667
Pune         2208
Lucknow      1936
Jaipur       1927
Kochi        1549
Name: count, dtype: int64
```

В результате WoE-кодирования по городам (на основе бинарной целевой переменной high_revenue) получили, что WoE положителен для Delhi (+0.0469) и Lucknow (+0.0871) в этих городах доля низкой выручки выше средней, а WoE отрицателен для Hyderabad (−0.0091), Mumbai (−0.0800) и Jaipur (−0.0155) здесь выше доля высокой выручки.


```
# Вывод таблицы WoE
print("WoE по группам городов:")
(woe_data[['city_group', 'count', 'events', 'non_events', 'WoE']])
```

WoE по группам городов:

	city_group	count	events	non_events	WoE
0	Bengaluru	3279	1476	1803	0.023423
1	Delhi	4042	1796	2246	0.046902
2	Hyderabad	2667	1222	1445	-0.009074
3	Jaipur	1927	886	1041	-0.015495
4	Kochi	1549	714	835	-0.020184
5	Lucknow	1936	841	1095	0.087149
6	Mumbai	4392	2090	2302	-0.080039
7	Pune	2208	1006	1202	0.001292

```
# Пример закодированных строк
print("\nПример закодированных строк:")
df[['city_group', 'avg_daily_revenue_inr', 'high_revenue', 'city_WoE']].head(10)
```

Пример закодированных строк:

	city_group	avg_daily_revenue_inr	high_revenue	city_WoE
0	Delhi	332.20	0	0.046902
1	Kochi	3138.90	1	-0.020184
2	Hyderabad	1626.36	1	-0.009074
3	Bengaluru	3592.98	1	0.023423
4	Delhi	2418.07	1	0.046902
5	Pune	1913.73	1	0.001292
6	Mumbai	1475.99	0	-0.080039
7	Lucknow	1004.87	0	0.087149
8	Bengaluru	1026.87	0	0.023423
9	Mumbai	2149.48	1	-0.080039

Бинарная целевая переменная `high_revenue` создана на основе медианы средней дневной выручки (1516.59 INR): 54.4% объектов - «низкая выручка» (0), 45.6% - «высокая выручка» (1).

WoE-кодирование по городам показало, что наиболее положительный WoE у Bengaluru (+0.0234) и Delhi (+0.0469) в этих городах относительно больше точек с низкой выручкой, а наиболее отрицательный WoE у

Hyderabad (−0.0091) и Mumbai (−0.0080) здесь выше доля точек с высокой выручкой.

Значения WoE близки к нулю, что указывает на слабую, но статистически значимую связь между городом и уровнем выручки.

В примере закодированных строк видно, что один и тот же город (например, Bengaluru) может иметь как высокую, так и низкую выручку, но его WoE-значение остаётся постоянным это корректное поведение при групповом кодировании.

Задача №23

Для набора данных для одного (произвольного) числового признака проведем обнаружение и удаление выбросов на основе правила трех сигм.

Выведем общую информацию о датасете:

```
# Общая информация о датасете
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22000 entries, 0 to 21999
Data columns (total 23 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   vendor_id                                22000 non-null   object
1   city                                     22000 non-null   object
2   zone_type                                22000 non-null   object
3   vendor_age_years                         22000 non-null   float64
4   years_in_business                       22000 non-null   float64
5   food_category                           22000 non-null   object
6   license_status                          22000 non-null   object
7   avg_daily_revenue_inr                   22000 non-null   float64
8   avg_daily_customers                     22000 non-null   float64
9   monthly_stall_rent_inr                  22000 non-null   float64
10  num_helpers                             22000 non-null   int64
11  hours_open_per_day                      22000 non-null   float64
12  competition_within_100m                 22000 non-null   float64
13  monthly_health_inspection_score          22000 non-null   float64
14  had_fine_last_year                      22000 non-null   float64
15  avg_monthly_rainfall_mm                 22000 non-null   float64
16  season_of_observation                    22000 non-null   object
17  has_online_presence                      22000 non-null   float64
18  customer_complaint_rate                  22000 non-null   float64
19  vendor_survived                         22000 non-null   int64
20  high_revenue                            22000 non-null   int32
21  city_group                              22000 non-null   object
22  city_WoE                                22000 non-null   float64
dtypes: float64(13), int32(1), int64(2), object(7)
memory usage: 3.8+ MB
```

Применим правило трёх сигм для обнаружения выбросов в числовом

признаке `avg_daily_customers` (среднее количество клиентов в день). Для этого вычисляем среднее значение и стандартное отклонение, определяем границы выбросов как интервал $[\text{mean} - 3 \cdot \text{std}, \text{mean} + 3 \cdot \text{std}]$, и находим все значения вне этого диапазона. В результате обнаружено 618 выбросов это около 2.81 % от общего объёма данных (22 000 строк). Такой уровень выбросов указывает на наличие аномальных наблюдений (например, нулевых или чрезмерно высоких значений клиентопотока), но не критичен для целостности датасета.

```
# Выбираем числовой признак
feature = 'avg_daily_customers'

# Вычисляем среднее и стандартное отклонение
mean = df[feature].mean()
std = df[feature].std()

# Определяем границы для выбросов
lower_bound = mean - 3 * std
upper_bound = mean + 3 * std

# Обнаружение выбросов
outliers = df[(df[feature] < lower_bound) | (df[feature] > upper_bound)]
print(f"Количество выбросов: {len(outliers)}")
```

Количество выбросов: 618

В результате удаления выбросов в числовом признаке `avg_daily_customers` по правилу трёх сигм получены следующие результаты: исходный размер датасета составлял 22000 строк, после удаления выбросов осталось 21382 строки, удалено 618 строк, что составляет 2.81% от исходного объёма.

```
# Удаление выбросов
df_cleaned = df[(df[feature] >= lower_bound) & (df[feature] <= upper_bound)]

# Проверка результатов
print(f"Исходный размер датасета: {len(df)}")
print(f"Размер датасета после удаления выбросов: {len(df_cleaned)}")
print(f"Процент удаленных строк: {100 * (len(df) - len(df_cleaned)) / len(df):.2f}%")
```

Исходный размер датасета: 22000

Размер датасета после удаления выбросов: 21382

Процент удаленных строк: 2.81%

Получаем очищенный датасет в виде пяти строк.

```
# Вывод первых 5 строк очищенного датасета
print("\nПервые 5 строк очищенного датасета:")
df_cleaned.head()
```

Первые 5 строк очищенного датасета:

	vendor_id	city	zone_type	vendor_age_years	years_in_business	food_category	license_status	avg_daily_revenue_inr	avg_daily_customers	monthly_stall_rent_inr	...	monthly_health_inspection_score	had_f
0	VSF-00001	Delhi	Industrial	34.0	10.98	Chinese	Licensed	332.20	10.0	0.00	...	65.00	
1	VSF-00002	Kochi	University Area	33.0	5.53	Chaat	Licensed	3138.90	180.0	2112.10	...	57.42	
2	VSF-00003	Hyderabad	Industrial	48.0	2.70	Desserts & Sweets	Licensed	1626.36	67.0	5487.75	...	57.42	
3	VSF-00004	Bengaluru	Transit Hub	37.0	9.77	Beverages	Licensed	3592.98	171.0	1960.79	...	36.38	
4	VSF-00005	Delhi	Transit Hub	51.0	21.24	South Indian	Licensed	2418.07	105.0	2720.82	...	71.05	

rows × 23 columns

Обнаружение и удаление выбросов в числовом признаке `avg_daily_customers` (среднее количество клиентов в день) по правилу трёх сигм дало следующие результаты:

- Обнаружено 618 выбросов (значения вне интервала $[\mu - 3\sigma, \mu + 3\sigma]$).
- Исходный размер датасета: 22000 строк.
- После удаления выбросов осталось: 21382 строки.
- Доля удалённых строк: 2.81% умеренная, что указывает на наличие некоторого числа аномальных значений (например, нулевых или сверхвысоких показателей клиентопотока), но не критичную для целостности выборки.

Таким образом, правило трёх сигм позволило очистить данные от экстремальных наблюдений.